



Desenvolvimento Front-End

Prof. Elias A. da Silva
elias.asilva@sp.senac.br

CSS3 – Flexbox

Nas últimas aulas o portfólio ganhou estrutura com HTML semântico e identidade visual com CSS. O que ainda falta é layout de verdade. Tudo está empilhado verticalmente porque é o comportamento padrão do navegador. Hoje isso muda. Flexbox é a ferramenta que resolve alinhamento e distribuição de elementos com muito menos código do que qualquer alternativa anterior.

Senac

CSS3 – Flexbox

ANTES — float + clearfix

```
.col { float: left; width: 33%; }  
.clearfix::after {  
  content: '';  
  display: table;  
  clear: both;  
}  
  
/* centralizar vertical era impossivel: */  
.box {  
  position: relative;  
  top: 50%;  
  transform: translateY(-50%);  
}
```



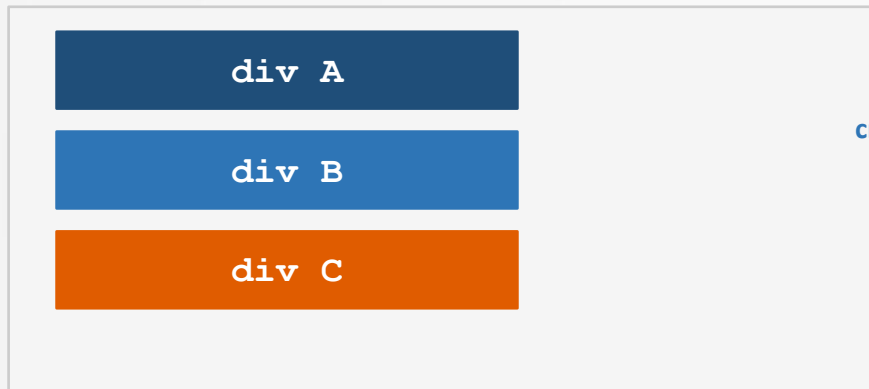
DEPOIS — Flexbox

```
.container {  
  display: flex;  
}  
  
/* centralizar vertical + horizontal: */  
.box {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Float foi a única forma de criar colunas por mais de 15 anos. Flexbox eliminou todos esses hacks. Hoje não existe motivo para usar float em layout.

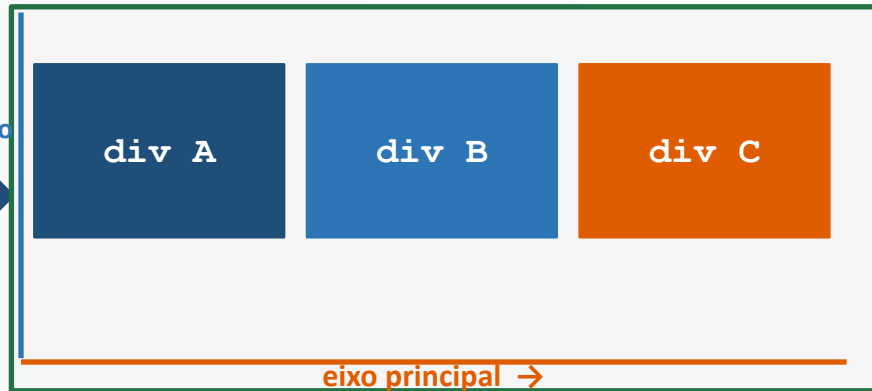
CSS3 – Display Flex

Sem Flexbox



display: flex

eixo
cruzado



eixo principal →

style.css

```
/* Uma linha transforma o layout */  
.container {  
  display: flex;    /* filhos viram flex items automaticamente */  
}  
  
/* Container controla COMO os filhos se distribuem */  
/* Cada item controla A SI MESMO dentro do container */
```

CSS – Display Flex

No style.css, adicionem o código do snippet slide 5.

Abram o portfólio e observem os links do nav. Já estavam lado a lado por causa do display inline-block que colocamos antes. Agora o nav é um flex container, o que nos dá controle muito mais preciso sobre o alinhamento.

Senac

CSS – Flex Direction

`flex-direction: row`

esquerda para direita (padrão)



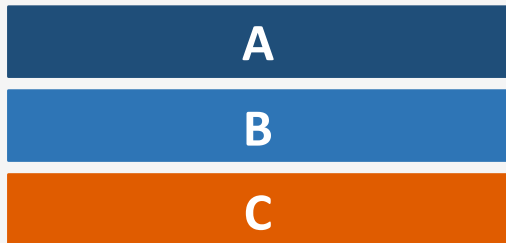
`flex-direction: row-reverse`

direita para esquerda



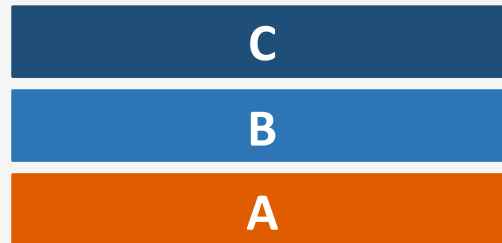
`flex-direction: column`

de cima para baixo



`flex-direction: column-reverse`

de baixo para cima



Quando muda para column, justify-content passa a controlar o eixo vertical e align-items o horizontal. Os eixos trocam de papel.

CSS – Flex Direction

Dentro da ultima adiç o ao style.css, header nav, coloquem flex-direction: row;

Adicionem column, vejam o efeito, depois voltem para row. Façam isso para fixar a troca de eixos antes de seguirmos.

Senac

CSS – justify-content

flex-start



flex-end



center



space-between



space-around



space-evenly



space-between — navbar: logo à esq, links à dir. **center** — centralizar sem margin auto. **space-evenly** — mais elegante em cards.

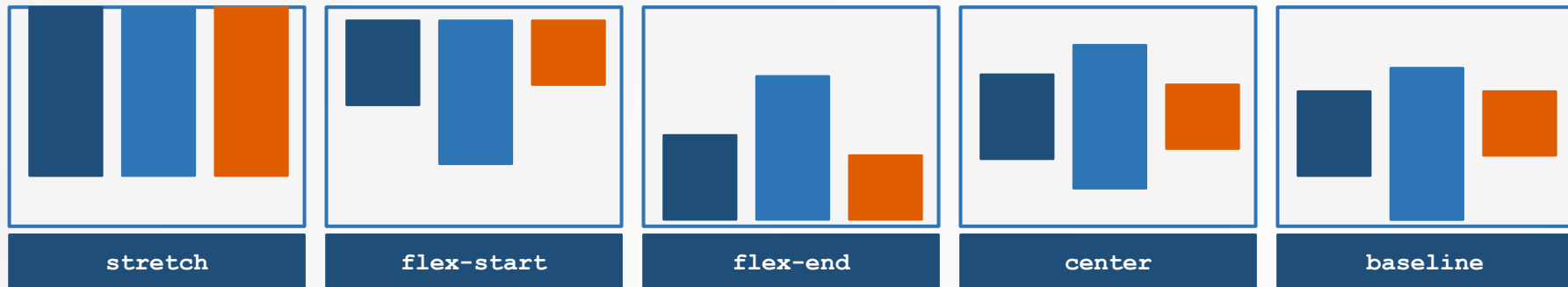
CSS – justify-content

Voltando ao style.css, sigam a direção do snippet 09;

Agora o h1 vai para a esquerda e o nav para a direita automaticamente. Isso é space-between funcionando. O align-items center verticalmente centraliza os dois em relação à altura do header.

Senac

CSS – align-items



`stretch` (padrão)

Todos os itens esticam para igualar a altura do maior. Por isso cards Flexbox ficam com a mesma altura automaticamente.

`center`

Centralização vertical perfeita. Resolve o maior problema histórico do CSS em uma linha. Use em heroes, modals e banners.

`flex-start` e `flex-end`

Alinha todos os itens no topo ou na base do container. Util em layouts onde a altura dos itens varia.

CSS – align-items

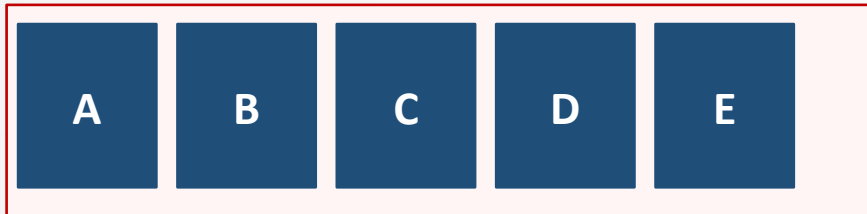
Voltando ao style.css, sigam a direção do snippet 11;

O conteúdo da seção hero agora está verticalmente centralizado dentro da section. O align-items flex-start mantém o texto alinhado à esquerda enquanto o justify-content center posiciona tudo no meio vertical.

Senac

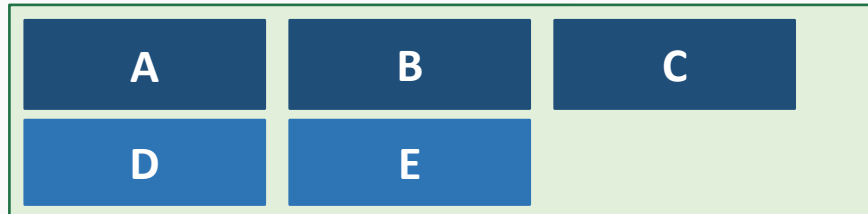
CSS – flex-wrap e gap

nowrap — sem quebra (padrão)



ítems encolhem para caber — quebra o layout em telas pequenas

wrap — quebra de linha



ítems quebram para a próxima linha quando não couberem

gap — espaçamento entre itens sem margin lateral

`gap: 16px;`

Igual em todas as direções

`gap: 16px 24px;`

16px linhas, 24px colunas

`row-gap: 16px;`

Só entre linhas

`column-gap: 24px;`

Só entre colunas

`style.css` — combinação essencial para grids de cards

```
.cards { display: flex; flex-wrap: wrap; gap: 24px; }
```

CSS – flex-wrap e gap

Voltando ao style.css, sigam a direção do snippet 13;

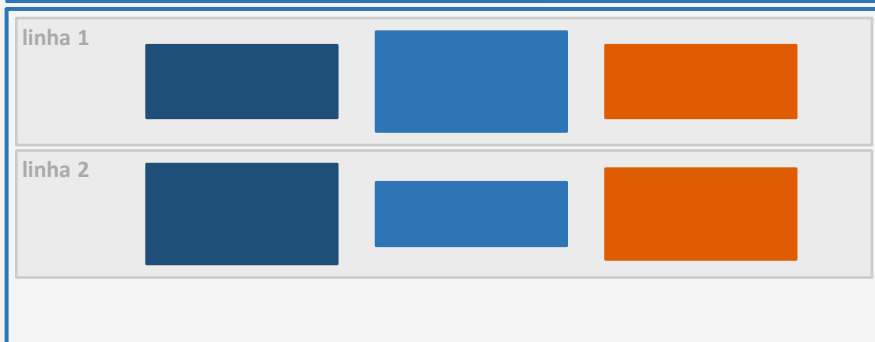
No HTML, adicionem `class='lista-habilidades'` na `ul` de habilidades. Os itens vão ficar como tags coloridas lado a lado, quebrando para a próxima linha quando necessário. Esse padrão de 'pill badges' aparece em todo portfólio profissional.

Atenção: Pode ser que haja erro nesse ponto, atenção para linha 179 no style.css, `nth-child` estará interferindo no que estamos fazendo, retire.

CSS – align-content

Só tem efeito quando **flex-wrap: wrap** está ativo E o container tem altura maior que o conteúdo E há mais de uma linha de itens.

align-items — dentro de cada linha



Cada linha alinha seus itens independentemente

align-content — entre as linhas



align-content: flex-start → linhas agrupadas no topo

Valores de align-content

flex-start

linhas agrupadas no topo

flex-end

linhas agrupadas na base

center

linhas centralizadas verticalmente

space-between

espaço entre as linhas, sem nas bordas

space-around

espaço igual ao redor de cada linha

stretch

linhas esticam para preencher (padrão)

CSS – Flex Items

flex-grow — distribui o espaço livre proporcionalmente

`grow: 1`

`grow: 2` — ocupa o dobro

`grow: 1`

flex-grow

0 (padrão) / número

Item não cresce (0) ou cresce ocupando espaço livre proporcional ao valor.

flex-shrink

1 (padrão) / 0

Item encolhe proporcionalmente (1) ou nunca encolhe (0). Use 0 em logos e ícones.

flex-basis

auto / tamanho

Tamanho base antes de grow e shrink. auto usa o width. 300px define 300px fixos.

flex (shorthand)

flex: grow shrink basis

flex: 1 = cresce igual e pode encolher.

flex: 1 1 280px = mínimo 280px, cresce livremente.

CSS – flex-items

Voltando ao style.css, sigam a direção do snippet 16, necessita atuar no html e depois no css;

O flex: 1 1 280px no card significa: cresça livremente, encolha se necessário, mas nunca fique menor que 280px. O flex-direction column no card com flex-grow 1 no parágrafo é o truque que mantém o botão sempre no fundo do card independente do tamanho do texto. Adicionem class='cards-projetos' na section de projetos e class='card-projeto' em cada article de projeto.

CSS – align-self e order

align-self — sobrescreve align-items só para um item

container: align-items: center

A
center

B
flex-start
align-self: flex-start

align-self: flex-end

C
flex-end

D
stretch

order — reordenação visual sem mudar o HTML

No HTML:

A — order: 2

B — order: 0

C — order: -1

Aparência visual:

C — order: -1

B — order: 0

A — order: 2

■ order muda a aparência, não o DOM. Leitores de tela seguem a ordem do HTML. Cuidado com acessibilidade ao reordenar conteúdo importante.

CSS – Prática – flexboxfroggy.com

- 1 24 desafios em ordem crescente de dificuldade.
- 2 Objetivo: mover os sapos para os nenúfares usando Flexbox.
- 3 Tente cada desafio por 2 minutos antes de pedir ajuda.
- 4 Busquem realizar os 24 desafios.

Joguem agora

Senac

4 Padrões Flexbox mais usados

Navbar: logo + links

```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Logo à esquerda, links à direita. O header universal.

Centralização perfeita

```
.hero {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Horizontal + vertical. O maior problema histórico do CSS resolvido em 2 linhas.

Cards: botão sempre no fundo

```
.card {  
  display: flex;  
  flex-direction: column;  
}  
.card p { flex-grow: 1; }
```

flex-grow no parágrafo empurra o botão para o fundo independente do texto.

Coluna principal + sidebar

```
.layout { display: flex;  
gap: 32px; }  
.main { flex: 3; }  
.sidebar { flex: 1; }
```

Proporção 3:1 sem calcular porcentagem. Responsivo com Media Query.

Esses 4 padrões cobrem 80% dos layouts que você vai construir na vida profissional.

CSS – Prática – Refatorar Portfólio

45 minutos — aplicar Flexbox em todo o portfólio agora

1 Header

```
header .container {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Logo à esquerda, nav à direita.

2 Hero

```
.hero {  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: flex-start;  
}
```

Conteúdo centralizado verticalmente.

3 Habilidades

```
.lista-habilidades {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 12px;  
}
```

Pill badges com quebra de linha.

4 Projetos

```
.cards-projetos {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 24px;  
}  
.  
card-projeto { flex: 1 1 280px; }
```

Cards fluidos, botão no fundo.

5 Formulário

```
.form-grid {  
  display: flex;  
  gap: 24px;  
}  
.  
form-campo { flex: 1; }
```

Campos nome e e-mail lado a lado.

6 Footer

```
footer {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Conteúdo centralizado.

CSS – O que aplicamos hoje



Header com logo e nav usando justify-content: space-between



Botão sempre no fundo via flex-grow no parágrafo



Hero com flex-direction: column e conteúdo centralizado



Campos do formulário em duas colunas com Flexbox



Habilidades como pill badges com flex-wrap e gap



Footer centralizado com Flexbox



Cards de projetos com flex: 1 1 280px



Flexbox Froggy — 24 desafios de fixação

Na Aula 07: CSS Grid Layout. Layouts bidimensionais. O que o Flexbox não resolve sozinho, o Grid resolve.

CSS – Tarefa Aula 06

- 1 Aplique Flexbox em ao menos um bloco do projeto da Atividade 1 que você está desenvolvendo.
- 2 Se o seu projeto tem cards ou seções lado a lado, esta aula deu o ferramental. Use agora.
- 3 Complete o Flexbox Froggy até o desafio 24 se ainda não chegou lá.
- 4 Revise o portfólio: header, cards e footer devem estar usando Flexbox corretamente.

Próxima aula — CSS Grid Layout. Traga o portfólio e o projeto da Atividade 1.